

Media Ratings Platform

Intermediate Hand-In

Student: Florian Gruber

Course: SWEN1 3C

Overview

The **Media Ratings Platform (MRP)** project is currently in an early development stage. The codebase compiles successfully, and the core architectural structure, including controllers, services, repositories, and models, has been established. However, the system is not yet fully functional.

Several essential components still need to be implemented or completed:

- The **PostgreSQL database** schema and tables have not yet been created or connected.
- Many **repository methods** are still placeholders and require real SQL query logic.
- The **business logic** is only partially implemented, with several service operations missing.
- **Authentication and token-based authorization** are planned but not fully integrated.
- **Unit and integration tests** have not been written yet and are necessary to verify functionality.
- **HTTP routes** are defined but not all are linked to working controller actions.

In its current state, the project serves as a **compiling prototype** — a working foundation of the architecture — but additional implementation, testing, and database setup are required before it can be considered operational.

```

classDiagram
    class Program {
        +Main() void
    }
    class HttpServer {
        +_listener : HttpListener
        +_router : RequestRouter
        +HttpServer()
        +Start() void
    }
    class Router {
        +_mediaController : Medi...
        +_ratingController : Ratin...
        +_userService : UserCo...
        +HandleRequest() : Task
        +RequestRouter()
        +WebsocketAsync() : Task
    }
    class AuthenticationService {
        +_token : Dictionary<string, string>
        +AuthenticationService()
        +Authorize() : bool
        +GenerateToken() : string
        +IsValidToken() : bool
    }
    class UserControler {
        +_auth : AuthenticationService
        +_service : UserService
        +handleRequestAsync() : Task
        +Login() : Task
        +Register() : Task
        +UserController()
    }
    class MediaEntryService {
        +_mediaRepo : MediaRepository
        +CreateMediaEntry(T>) : MediaEntry
        +MediaEntryService()
    }
    class UserService {
        +_userRepository : UserRepository
        +CheckPassword() : bool
        +CheckUserData() : bool
        +CreateUser() : void
        +LoginUser() : void
        +UserService()
    }
    class MediaEntry {
        <<Abstract Class>>
        +AgeRestriction : int
        +CreatedAt : DateTime
        +Creator : Guid
        +Description : string
        +Id : Guid
        +ReleaseYear : DateOnly
        +Title : string
        +MediaEntry() (+ 1 Überladung)
    }
    class Series {
        <<Abstract Class>>
        +Genre : HashSet<SeriesGenre>
        +Series() (+ 1 Überladung)
    }
    class Movie {
        <<Abstract Class>>
        +Genre : HashSet<MovieGenre>
        +Movie() (+ 1 Überladung)
    }
    class Game {
        <<Abstract Class>>
        +Genre : HashSet<GameGenre>
        +Game() (+ 1 Überladung)
    }
    class MediaRating {
        +_usersWhichLiked : HashSet<Guid>
        +Comment : string
        +CreatedAt : DateTime
        +Entry : Guid
        +Id : Guid
        +IsPublic : bool
        +Likes : int
        +Stars : StarRating
        +LikeFrom() : void
        +MediaRating() (+ 1 Überladung)
    }
    class User {
        +FavoriteEntryIds : HashSet<Guid>
        +Id : Guid
        +Password : string
        +Username : string
        +User() (+ 1 Überladung)
    }
    class IRepository_TModel {
        <<Generic Interface>>
        +Add() : bool
        +DeleteById() : bool
        +GetAll() : IEnumerable<TModel>
        +GetById() : TModel
        +Update() : bool
    }
    class MediaRepository {
        +_connString : string
        +Add(T>) : void (+ 1 Überladung)
        +DeleteById() : void
        +GetAll() : IEnumerable<MediaEntry>
        +GetById() : MediaEntry
        +IRepository_MediaEntry.DeleteById(...) : MediaRepository
        +IRepository_MediaEntry.Update(...) : MediaRepository
        +Update() : void
    }
    class UserRepository {
        +_connString : string
        +Add() : bool
        +DeleteById() : bool
        +GetAll() : IEnumerable<User>
        +GetById() : User
        +Update() : bool
        +UserRepository()
    }
    Program --> HttpServer
    HttpServer --> Router
    Router --> AuthenticationService
    Router --> MediaEntryService
    Router --> UserService
    Router --> MediaEntry
    Router --> Series
    Router --> Movie
    Router --> Game
    Router --> MediaRating
    Router --> User
    AuthenticationService --> MediaEntry
    AuthenticationService --> Series
    AuthenticationService --> Movie
    AuthenticationService --> Game
    AuthenticationService --> MediaRating
    AuthenticationService --> User
    MediaEntryService --> MediaEntry
    MediaEntryService --> Series
    MediaEntryService --> Movie
    MediaEntryService --> Game
    MediaEntryService --> MediaRating
    MediaEntryService --> User
    UserService --> MediaEntry
    UserService --> Series
    UserService --> Movie
    UserService --> Game
    UserService --> MediaRating
    UserService --> User
    MediaEntry --|> Series
    MediaEntry --|> Movie
    MediaEntry --|> Game
    MediaEntry --|> MediaRating
    MediaEntry --|> User
    
```

At the top level, the **controllers** act as the API interface — they handle incoming HTTP requests, parse input data using the Newtonsoft.Json Dependency, call the appropriate business services, and return responses to the client.

authorization. Services coordinate between repositories and models without knowing anything about HTTP or databases.

The **repositories** handle all direct interaction with the PostgreSQL database. They execute SQL queries using Npgsql, map the results into domain model objects, and abstract away data persistence details.

The **models** represent the core domain entities, such as users, media entries, and ratings. They define the data and behavior relevant to the business domain, for example methods to rate media or toggle visibility.

DTOs (Data Transfer Objects) are used to transfer data between the API and the client. They are lightweight, serializable objects that expose only the necessary fields to the outside world, hiding internal implementation details.

The **database** stores all persistent data — users, media, ratings, and relationships like favorites or genre assignments — in normalized relational tables. Enums such as media type or genre are stored as text or integer values.

Finally, the **server** acts as the entry point of the program. It configures the HTTP listener, registers controllers and routes, initializes services and repositories, and starts handling requests on a given port.

Git Repository

Repository Link: <https://github.com/flogru03/MRP>